

Reproducibility

Table A1: All hyperparameters of our training setup.

Parameter	value
Model	
Parameter count	268M
Transformer block count	16
Attention head count	8
Embedding dimension	1024
Activation	SwiGLU (widening factor 2.66)
Sequence length	81 (or 76 with no best move prediction)
Masking schedule	linear
Tokenizer	
Vocabulary size	52 (+ 1 mask token)
Theme count	66 (8 are not checked for in theme criterion)
Supervised training	
Optimizer	AdamW
Maximum learning rate	$3 \cdot 10^{-4}$
Learning rate schedule	linear + cosine annealing
Weight decay	10^{-4}
Gradient norm clip	1.0
Batch size	1024
Steps	2,000,000
GPUs	8x Nvidia H200
Reinforcement learning	
Optimizer	AdamW
Learning rate	$3 \cdot 10^{-5}$
β, γ, ϵ	0.1, 0.2, 0.2
Num groups	6
Group size	16
Denoising steps	128
Temperature	1.0
Theme distribution	Lichess dataset
PPO epochs	1
GPUs	1x Nvidia H200
FEN dist limit	6
PV dist limit	1
Model entropy limit	0.6
Metrics	
$\tau_{\text{uni}}, \tau_{\text{cnt}}$	0.5, 0.1
Stockfish version	dev-20260111-eb5a65ae (dev build from just before SF 18)

In Table A1, we present the hyperparameters of our model training. For the diffusion model’s denoising network, we employ a transformer encoder with 268M parameters. We base our model architecture on the work of (Ruoss et al. 2024), who utilize a 270M parameter transformer for chess. We employ 16 transformer blocks with an embedding dimension of 1024. Additionally, we employ 8 attention heads in each attention layer. This base model results in around 200M

parameters. The total parameter count is further increased by our conditioning approach, which is implemented via cross attention in each transformer block. The conditioning accounts for the 68M parameters. With this architecture, we train two different models. As we experiment with best move prediction, we train one model that can predict the best move and one that cannot. This results in a minor decrease in parameters in positional embedding, as the sequence length of the model that predicts the best move has a sequence length of 81, whereas, the other model has a sequence length of 76.

As we condition the generative process on puzzle themes, our models input a one-hot encoding of the themes. We choose themes based on the Lichess puzzler project (Dupleissis 2020), which implements conditions for each theme. To represent a chess board in machine-readable form, we use the Forsyth-Edwards notation (FEN) (Chess.com 2020), which we tokenize in a similar manner as (Ruoss et al. 2024; Feng et al. 2026). They pad the FEN into a fixed length sequence of characters, which are converted into tokens. We modify this procedure slightly by using separate tokens to represent different concepts. For instance, a black bishop and black-to-move are given separate tokens in our procedure even though they are both represented by the character b. Additionally, the best move is tokenized in the universal chess interface (UCI) format and concatenated into the FEN for the best move model.

The supervised training is conducted by minimizing the evidence lower bound with gradient descent. We employ the AdamW optimizer (Loshchilov and Hutter 2019) and a learning rate schedule, which starts with a linear warmup for 1,000 steps and proceeds with cosine annealing until the 2,000,000 steps have been reached. The linear warmup makes the learning rate go from $3 \cdot 10^{-6}$ to $3 \cdot 10^{-4}$ and the cosine annealing takes it from $3 \cdot 10^{-4}$ to $3 \cdot 10^{-5}$. We use a small weight decay of 10^{-4} and clip the gradient norm at 1.0. The total batch size is 1024 positions. Finally, the training is conducted on a single node of eight Nvidia H200 GPUs.

Lastly, for the RL phase, we continue utilizing the AdamW optimizer but with a reduced learning rate of $3 \cdot 10^{-5}$, executing the training on a single Nvidia H200 GPU. During generation, we sample the themes from the theme distribution of the Lichess dataset. During the trajectories, we employ 128 denoising steps and a temperature of 1.0. To perform the policy updates, we use 6 groups of size 16 to compute the advantages. Diversity and realism are maintained with $\beta = 0.1$ and $\gamma = 0.2$. We ensure stable updates with the PPO clip parameter $\epsilon = 0.2$. The reward function evaluates the generated positions using the latest pre-release development build of Stockfish (dev-20260111-eb5a65ae) at the time of the start of this work. In the reward function, positions are considered too close if their board distance is below 6 or their PV distance is below 1. This means that positions are considered different in terms of the PV distance, if the best moves differ even slightly. Finally, to prevent the policy from collapsing, we also enforce a model entropy limit of 0.6.

Theme computation

To identify the tactical themes present in a generated position, we employ the Lichess puzzler pipeline, which iterates

Category	Themes
State-of-game	Opening, Middlegame, Endgame
Type-of-endgame	Pawn endgame, Bishop endgame, Knight endgame, Rook endgame, Queen endgame, Queen rook endgame
Type-of-checkmate	Mate, Back rank mate, Boden mate, Smothered mate, Hook mate, Double bishop mate, Arabian mate, Dovetail mate, Anastasia mate
Length-of-checkmate	Mate in 1, Mate in 2, Mate in 3, Mate in 4, Mate in 5
Length-of-puzzle	One move, Short, Long, Very long
Winning	Crushing, Advantage
Other	Hanging piece, Fork, Interference, Kingside attack, Zugzwang, Exposed king, Skewer, Pin, Quiet move, Discovered attack, Sacrifice, Deflection, Advanced pawn, Attraction, Promotion, Queenside attack, Defensive move, Attacking f2/f7, Clearance, Intermezzo, Equality, Trapped piece, X-ray attack, Capturing defender, Double check, En passant, Castling, Underpromotion

Table A2: The complete list of themes supported by the Lichess puzzler pipeline, grouped by their respective categories.

Algorithm 1: Themes match

Require: Requested themes $\mathcal{T}_{\text{req}}$, Generated puzzle themes $\mathcal{T}_{\text{gen}}$

Require: Category sets $\mathcal{S}_{\text{state}}$, $\mathcal{S}_{\text{endgame}}$, $\mathcal{S}_{\text{mate_len}}$, $\mathcal{S}_{\text{mate_type}}$, $\mathcal{S}_{\text{other}}$

```

if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{state}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
  return False
end if
if "endgame"  $\in \mathcal{T}_{\text{req}}$  then
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{endgame}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
end if
if "mate"  $\in \mathcal{T}_{\text{req}}$  then
  if "mate"  $\notin \mathcal{T}_{\text{gen}}$  then
    return False
  end if
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{mate\_len}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{mate\_type}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
else
  if  $(\mathcal{T}_{\text{req}} \cap \mathcal{S}_{\text{other}}) \not\subseteq \mathcal{T}_{\text{gen}}$  then
    return False
  end if
end if
return True

```

over the principal variation to detect specific themes. We group these themes into several distinct categories, which include the following: state of game, type of endgame, type of checkmate, length of checkmate, length of puzzle, winning states, and other tactics. Table A2 enumerates every specific theme we evaluate during the puzzle evaluation process. While the Lichess puzzle dataset contains additional metadata tags, we exclusively focus on objective, position-based themes. We intentionally discard the other tags, such as "master vs. master", as they cannot be verified from the board state alone.

During reinforcement learning and evaluation, we determine whether a generated puzzle successfully matches the requested conditions by comparing the generated theme set $\mathcal{T}_{\text{gen}}$ against the requested theme set $\mathcal{T}_{\text{req}}$. Rather than requiring a strict superset match $\mathcal{T}_{\text{req}} \subseteq \mathcal{T}_{\text{gen}}$, we use the specific groups for the check. The complete theme matching logic is detailed in Algorithm 1.

Conditioning on difficulty ratings

In addition to the puzzle themes, we condition the generation on puzzle difficulty Elo ratings. We, however, do not observe a substantial effect of the rating on the final puzzle rate compared to the themes. This happens because the themes and the rating are correlated, and the model infers the difficulty from the themes. For instance, a mate-in-one puzzle would never have a high difficulty rating in the Lichess dataset, which the model is able to infer from the theme. The effect is further amplified by RL, where the model is rewarded when it generates difficult puzzles with correct themes. The target rating is not part of our reward function, so it is essentially forgotten during RL. In Figure A1, we present the feature importance of the themes and the rating, calculated using a random forest model to predict the puzzle rate. We observe that after supervised training, the themes explain much more of the puzzle rate uncertainty than the Elo rating. The effect is even greater after RL, as expected, where themes account

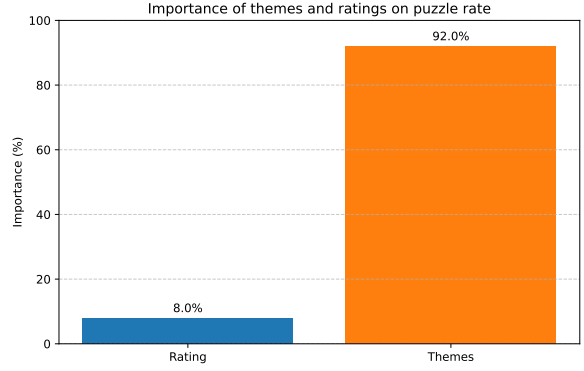
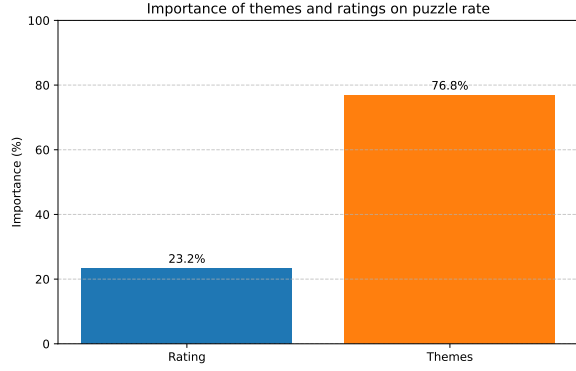


Figure A1: The impurity based importance of themes and rating on the puzzle rate after supervised training (left) or after RL (right).

for over 90% of the model’s ability to predict the puzzle rate.

Hierarchical generation

To further experiment with partial board conditioning, we use our model to generate new positions hierarchically. In a standard masked diffusion reverse process, all tokens are unmasked gradually over the total number of time steps to form a single final state. In our hierarchical setup, we instead define a set of specific time steps at which we split the sampling process. We run the denoising process on a single state until a predetermined split step is reached. At this juncture, the partially unmasked state, which still contains masked tokens, is duplicated into multiple independent branches. Each branch then continues its own reverse process. Because the remaining masked tokens are sampled independently in each branch, they resolve into different final piece configurations while retaining the shared tokens that were unmasked prior to the split.

Figure A2 illustrates how this generative process diverges from a common starting point. After sampling the initial themes, we begin the sampling process by splitting the initial state into four trajectories at step 256. We then apply the reverse process to each branch until the next predetermined split point at step 213, where we branch the generation again. Three of these four initial branches are shown on level two of Figure A2. We repeat this alternating procedure of splitting and denoising across specific time steps for a total of six iterations, ultimately ending up with $4^6 = 4096$ positions. From this batch, we select a few interesting candidates to visualize in each subtree. Interestingly, we observe that the majority of the diversity between puzzles is established early in the generative process, while the latter splits mostly end up in similar puzzles. If we look at positions where the latest common ancestor is at step 128, with approximately half of the tokens masked, most puzzles share very similar principal variations. This likely occurs because critical components, such as kings, rooks, and important pawns, have already been unmasked by this stage, leaving minimal room to vary the best move.

This hierarchical generation strategy highlights a structural difference of masked diffusion models over standard autoregressive sequence models. While a similar sampling method can be applied to autoregressive models, a major disadvantage is that the shared components between branches are strictly constrained to a sequential prefix. Because autoregressive models generate tokens in a fixed order, any shared ancestry dictates that the beginning of the string remains identical across all branches, forcing all variation into the later tokens. In the context of chess puzzle generation, this means the generated variations would always share the same pieces on the starting ranks, which will not appear random to a human eye. In contrast, our masked diffusion model unmask tokens globally based on a masking schedule rather than their position in the sequence. Consequently, the shared tokens of a subtree can be distributed anywhere on the board, allowing the hierarchical variations to appear naturally random to human observers.

Reinforcement learning entropy collapse

As discussed previously, our reinforcement learning setup suffers from entropy collapse during extended training. In Figure A3, we observe two common positions after the collapse. In the position on the left, we see a checkmate in two, where a knight is sacrificed. On the other hand, on the right, we observe a checkmate in three, where after a queen check, White sacrifices the queen on e8 to obtain back-rank checkmate. What is common to both of these positions is that the exact positioning of the White queen is not important. The White queen can be on any square, which sees h7 on the left and e7 on the right. Similarly, the White rook can be on any square on the e-file on the right position. This observation helps us understand the collapse, as two similar positions may not be caught immediately by the diversity filtering. Two such positions may pass, if the board distance is high enough controlled by the board’s queen side and if the PV distance is nonzero controlled by the initial positioning of the queen.

Step 256

Rook Endgame Generation

Themes: crushing, long, endgame, rook endgame, advanced pawn

Step 213

Step 171

Step 0

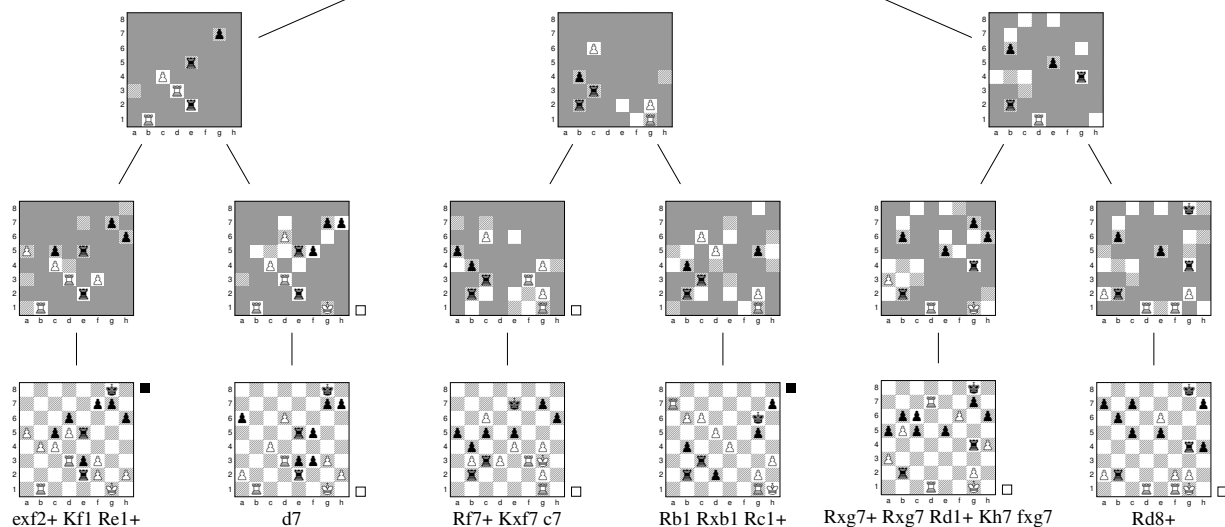


Figure A2: The hierarchical generation process of a rook endgame. In predetermined steps, we split the generative process in two and obtain a tree of positions for a single set of themes.

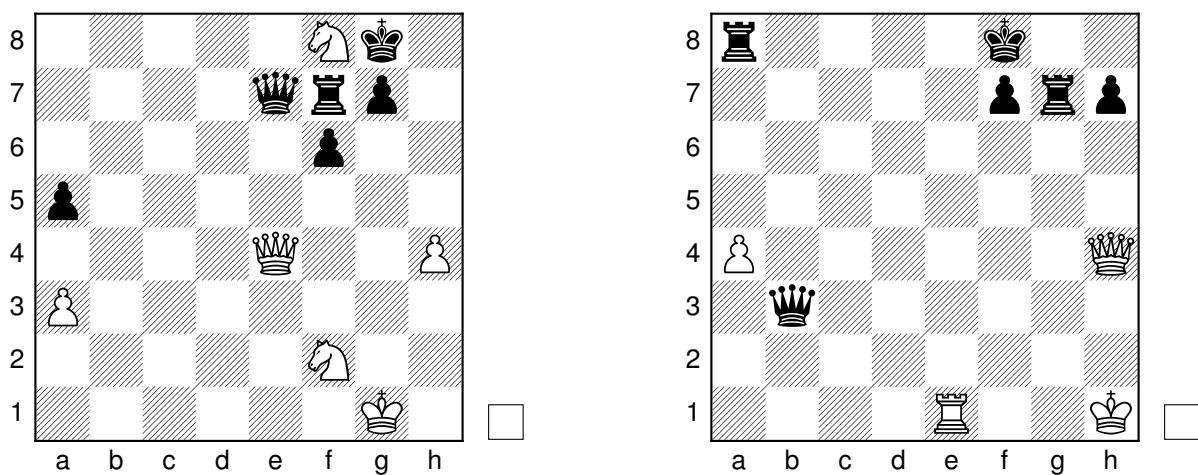


Figure A3: Example positions from the model after it has collapsed during RL. Neither of the positions are immediately ruled out by diversity filtering, as the initial positioning of the White queen is not important for the puzzle. Both positions lead to a checkmate via a sacrifice. The solutions of the positions are as follows: (left) Qh7+ Kxf8 Qh8# (right) Qe7+ Kg8 Qe8+ Rxe8 Rxe8#.